Autor: Jakub Staniszewski, Jacek Łoboda
Data: 16 marca 2026 r.
Laboratorium nr: 1
Temat lab: Analiza błędów

1. Treść zadań

- **Zadanie 1:** Oblicz przybliżoną wartość pochodnej funkcji, używając wzoru na różnicę progresywną $f'(x) \approx \frac{f(x+h)-f(x)}{h}$. Sprawdź działanie programu dla funkcji $\tan(x)$ w punkcie $x = 1$. Powtórz ćwiczenie, używając wzoru różnic centralnych. Na wspólnym rysunku przedstaw wykresy wartości bezwzględnej błędu metody, błędu numerycznego oraz błędu obliczeniowego w zależności od kroku h.
- **Zadanie 2:** Przepisz podane wyrażenia, tak aby zmniejszyć błąd numeryczny dla wskazanych argumentów.
- **Zadanie 3:** Rozważana jest funkcja $f(x) = \frac{e^x-1}{x}$ dla $x \neq 0$ oraz $f(x) = 1$ dla $x = 0$. Zaproponuj sposób obliczania funkcji o lepszych własnościach numerycznych, korzystając z rozwinięcia w szereg Taylora oraz funkcji `expm1` .
- **Zadanie 4:** Napisz program obliczający sumę n liczb zmiennoprzecinkowych pojedynczej precyzji, losowo rozłożonych w przedziale [0,1]. Sumę należy obliczyć na 5 sposobów, wykorzystując m.in. różne typy akumulatorów, sortowanie elementów oraz algorytm Kahana z kompensacją błędu.

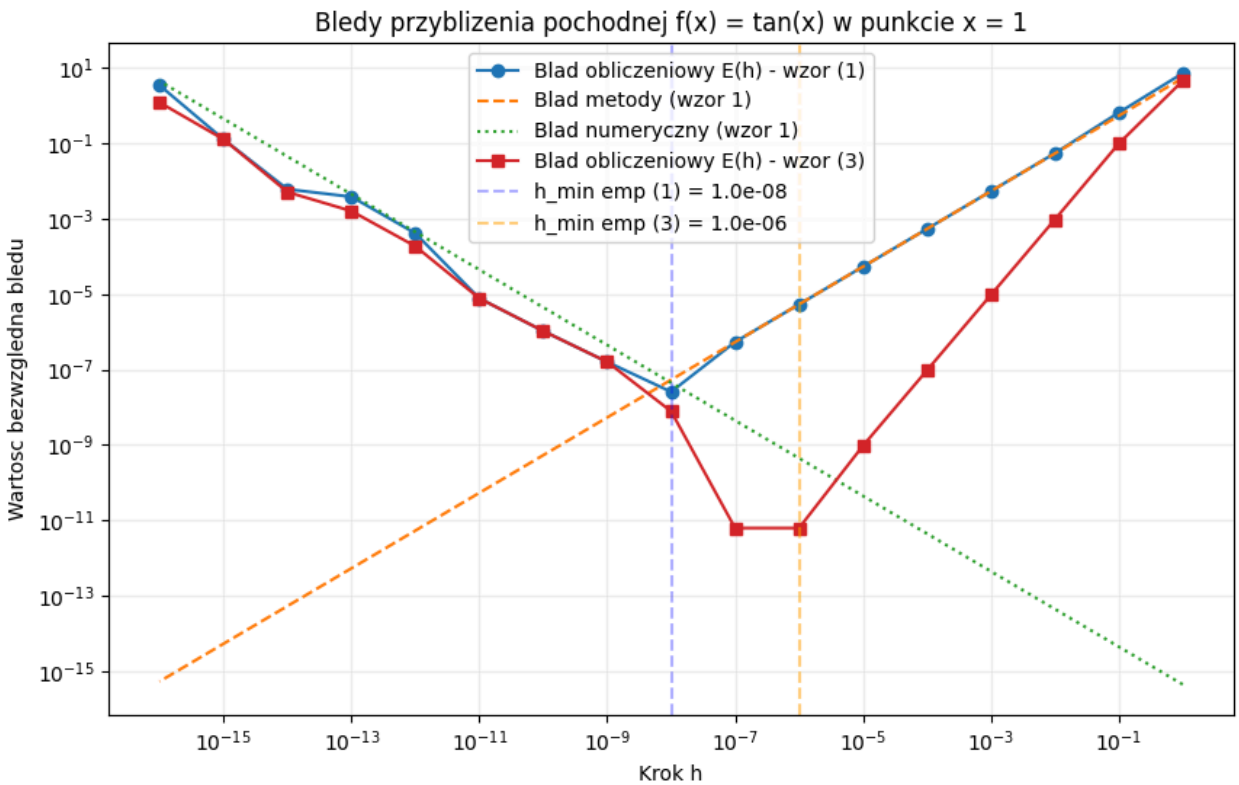
2. Argumentacja rozwiązania zadań i fragmenty algorytmu

Zadanie 1

Pochodna numeryczna jest obarczona dwoma rodzajami błędów: błędem metody (wynikającym z ucięcia szeregu Taylora) rosnącym wraz z h oraz błędem numerycznym (błędem zaokrągleń) rosnącym przy bardzo małych h z powodu utraty cyfr znaczących przy odejmowaniu zbliżonych wartości. Oczekujemy wykresu w kształcie litery "V", gdzie dno określa minimalny możliwy błąd.

```
for k, h in zip(k_vals, h_vals):
    df_fw = (f(x + h) - f(x)) / h
    err_1 = np.abs(df_fw - exact)

    df_cd = (f(x + h) - f(x - h)) / (2.0 * h)
    err_2 = np.abs(df_cd - exact)
```



Rys. 1. Wykres zależności wartości bezwględnej błędu od wartości błędu h.

Wnioski:

Wykres błędu posiada wyraźne minimum. Osiągnięcie najniższego błędu waha się na granicy wpływu błędu obcięcia oraz utraty precyzji obliczeniowej. Metoda różnic centralnych jest weryfikowalnie znacznie dokładniejsza (rząd rzędu $\approx 10^{-11}$) niż metoda progresywna (rząd $\approx 10^{-8}$).

Zadanie 2

Poniżej przedstawiono przekształcenia mające na celu zmniejszenie błędu numerycznego dla wskazanych argumentów.

- $\sqrt{x+1}-1$, $x \approx 0$

$$\frac{(\sqrt{x+1}-1)(\sqrt{x+1}+1)}{\sqrt{x+1}+1} = \frac{x+1-1}{\sqrt{x+1}+1} = \frac{\mathbf{x}}{\sqrt{\mathbf{x+1}}+\mathbf{1}}$$

- $x^2 - y^2$, $x \approx y$

$$(\mathbf{x}-\mathbf{y})(\mathbf{x}+\mathbf{y})$$

- $\frac{1-\cos x}{\sin x}$, $x \approx 0$

$$\frac{2\sin^2(x/2)}{2\sin(x/2)\cos(x/2)} = \frac{\sin(x/2)}{\cos(x/2)} = \tan(\mathbf{x/2})$$

- $\sin x - \sin y$, $x \approx y$

$$\mathbf{2}\sin\left(\frac{\mathbf{x}-\mathbf{y}}{\mathbf{2}}\right)\cos\left(\frac{\mathbf{x}+\mathbf{y}}{\mathbf{2}}\right)$$

- $\frac{a+b}{2}$ (bisekcja odcinka $[a,b]$)

$$\mathbf{a} + \frac{\mathbf{b}-\mathbf{a}}{\mathbf{2}}$$

- $\mathrm{softmax}(x)_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$
  $M = \max(x)$

$$\frac{\exp(\mathbf{x_i}-\mathbf{M})}{\sum_{\mathbf{j}}\exp(\mathbf{x_j}-\mathbf{M})}$$

Oryginalna	Stabilna	Argumenty	Wartość_og	Wartość_st
$\sqrt{\mathbf{x+1}}-\mathbf{1}$	$\frac{\mathbf{x}}{\sqrt{\mathbf{x+1}}+\mathbf{1}}$	$\mathbf{x} = \mathbf{10^{-11}}$	$\mathbf{0.00000}$	$\mathbf{5E-12}$
$\mathbf{x^2}-\mathbf{y^2}$	$(\mathbf{x}-\mathbf{y})(\mathbf{x}+\mathbf{y})$	$\mathbf{x} = \mathbf{1.0000000001}$ $\mathbf{y} = \mathbf{1.0000000000}$	$\mathbf{0E-9}$	$\mathbf{2E-13}$

Tabela 1. Różnice w wartościach oryginalnej i stabilnej wersji wyrażeń na przykładzie dwóch podpunktów, precyzja wynosi 10 cyfr znaczących.

Wnioski:

- Nawet poprawna matematycznie metoda może produkować błędne wyniki w arytmetyce komputerowej, dlatego postać wzoru jest kluczowa dla stabilności.
- Błąd kancelacji można skutecznie wyeliminować poprzez przekształcenia algebraiczne, takie jak mnożenie przez sprzężenie lub faktoryzacja.
- Otrzymanie wyniku 0.00000 lub 0E-9 w wersji oryginalnej przy niezerowym wyniku wersji stabilnej dowodzi utraty informacji o wyniku na skutek zaokrągleń.

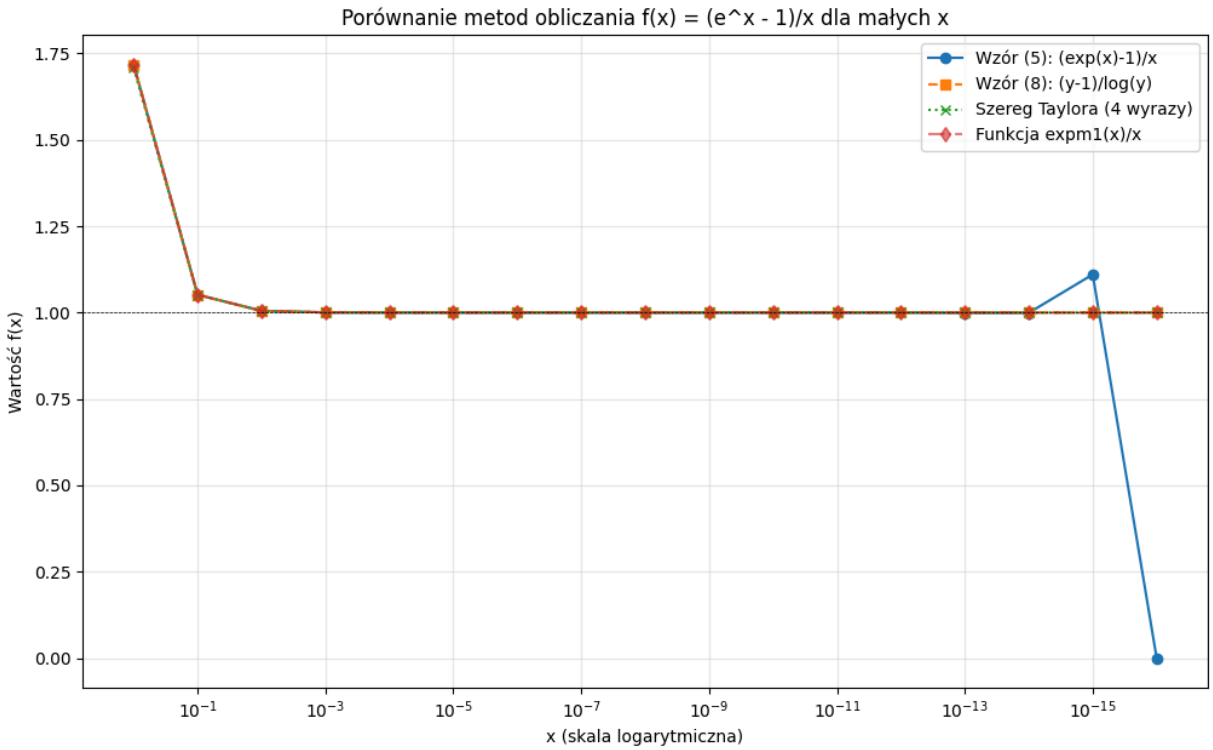
Zadanie 3

Dla małych x odejmowanie $e^x - 1$ skutkuje błędami zaokrągleń. Zastosowanie szeregu Taylora wyodrębnia wyrazy sprawiające problem. Dedykowana funkcja biblioteczna `expm1` realizuje to zadanie w tle i podnosi stabilność.

```
def f1(x):
    if x == 0:
        return 1.0
    return (np.exp(x) - 1.0) / x

def f3(x):
    return 1.0 + x/2.0 + (x**2)/6.0 + (x**3)/24.0

def f4(x):
    if x == 0:
        return 1.0
    return np.expm1(x) / x
```



Rys. 2. Wykres zależności wartościfunkcji od jej argumentu.

Wnioski:

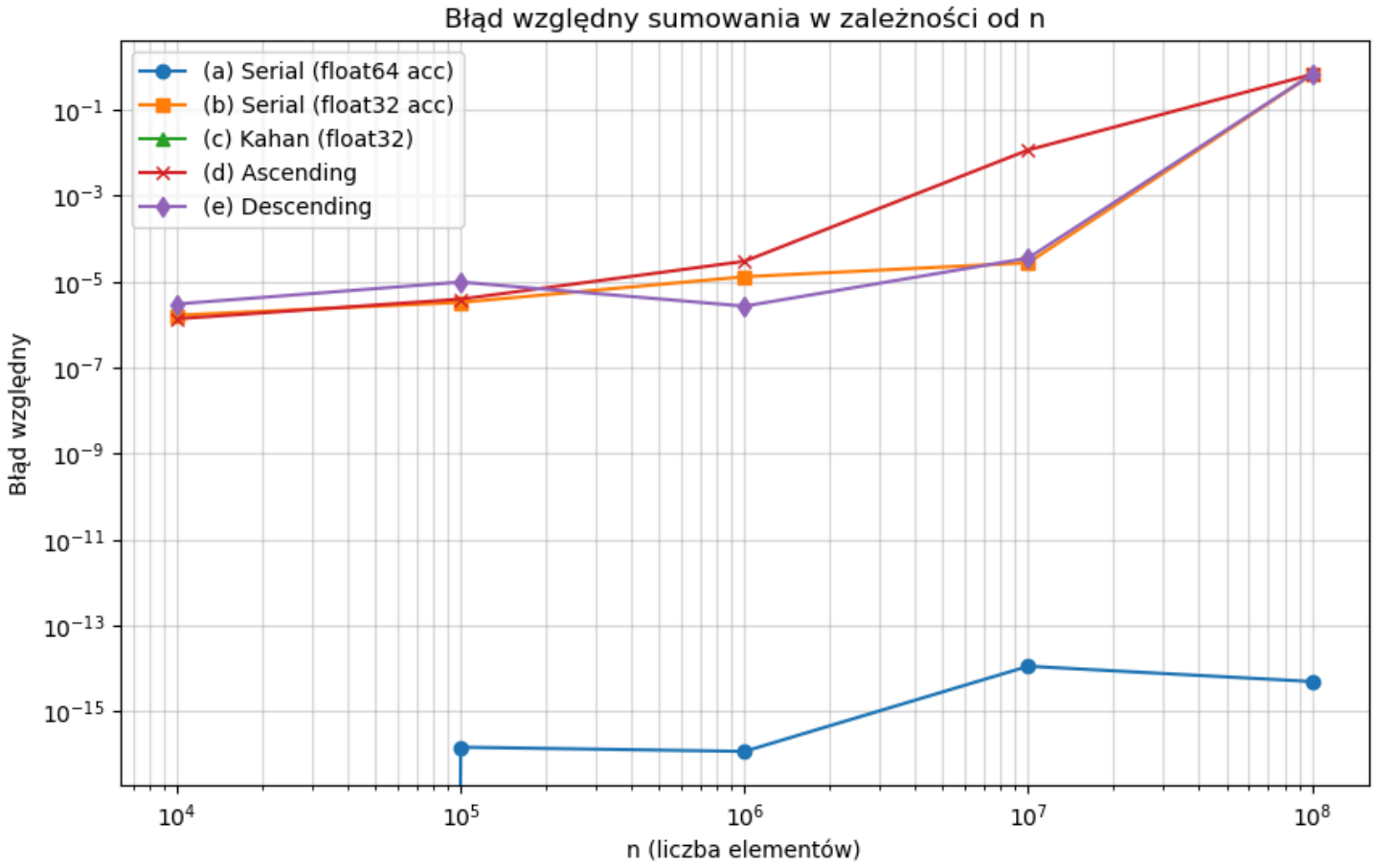
Powszechne biblioteczne metody numeryczne ulegają znaczącej degradacji stabilności, gdy stosowane są blisko wartości granicznych (jak ułamek w rejonie bliskim zeru). Dedykowane transformacje (np. szereg Taylora lub optymalizowane niskopoziomowo instrukcje typu `expm1`) całkowicie stabilizują obszar numerycznej "niepewności".

Zadanie 4

Celem ćwiczenia było porównanie pięciu metod sumowania n liczb typu float32 z przedziału [0, 1] względem wyniku wzorcowego z math.fsum()

Metody:

- Sumowanie losowe z akumulatorem float64
- Sumowanie losowe z akumulatorem float32
- Algorytm Kahana (kompensacja błędu), float32
- Sumowanie po posortowaniu rosnąco, float32
- Sumowanie po posortowaniu malejąco, float32



Rys. 3. Wykres zależności zaobserwowanego błędu sumowania od liczby sumowanych elementów.

Wnioski:

- Algorytm Kahana (c) za każdym razem daje najdokładniejszy wynik
- Zastosowanie akumulatora float64 (a) znacząco redukuje błąd względem metod 32-bitowych
- Standardowe metody (b, d, e) wykazują zbliżony poziom błędów przy niskiej precyzji obliczeń

5. Bibliografia

1. Materiały dydaktyczne do laboratorium: *Analiza błędów* (lab1.pdf)
2. Oficjalna dokumentacja pakietu `NumPy` oraz wizualizacje za pomocą biblioteki `Matplotlib`
3. Dokumentacja standardowej wbudowanej biblioteki `decimal`